

Mystery Tourist Lens

Tecnologías y reparto de trabajo

Proyecto de Sistemas Multimedia basado en APIs de Google Cloud
WebApp/PWA turística gamificada con inteligencia artificial
Grupo de 3 personas

1. Objetivo técnico del proyecto

Mystery Tourist Lens será una **WebApp/PWA mobile-first**. El usuario podrá abrir la aplicación desde el navegador del móvil, instalarla como si fuera una app, hacer o subir una foto de un monumento, recibir una audioguía generada con IA y resolver un enigma para desbloquear el siguiente punto de la ruta.

El objetivo técnico es mantener una arquitectura sencilla:

- El **frontend** se encarga de la experiencia de usuario.
 - El **backend** se encarga de llamar a las APIs de Google Cloud de forma segura.
 - La **base de datos** guarda progreso, puntuación y puntos desbloqueados.
 - La **ruta demo** tendrá 3 o 4 puntos de interés para que el MVP sea controlable.
-

2. Stack tecnológico recomendado

Frontend

- **Vue 3**: framework principal para crear la WebApp/PWA.
- **Vite**: herramienta de desarrollo y build rápida para Vue.
- **TypeScript**: código más ordenado y menos errores en componentes, datos y llamadas API.
- **Vue Router**: navegación entre pantallas: inicio, cámara, audioguía, enigma, mapa y progreso.
- **Pinia**: estado global de la app: punto actual, puntuación, progreso y datos del usuario.
- **Ionic Vue**: componentes móviles ya preparados: botones, cards, tabs, modales, alertas, loaders y navegación con aspecto de app.
- **vite-plugin-pwa**: manifest, service worker e instalación de la PWA en el móvil.
- **Axios**: llamadas HTTP desde el frontend al backend.
- **@googlemaps/js-api-loader**: carga de Google Maps en la pantalla del mapa.

Backend

- **Node.js**: entorno de ejecución del backend.
- **Express**: API REST sencilla con varios endpoints.
- **Multer**: recepción de imágenes subidas desde el frontend.
- **CORS**: permitir peticiones desde la WebApp al backend.
- **dotenv**: variables de entorno en desarrollo local.
- **Google Cloud SDKs para Node.js**: conexión con Vision, Text-to-Speech, Translation, Storage, Firestore y Vertex AI/Gemini.
- **Google Cloud Run**: despliegue recomendado del backend.

Se recomienda **Cloud Run** en lugar de varias Cloud Functions porque permite tener todos los endpoints en un único backend Express, lo que simplifica el desarrollo y las pruebas.

Google Cloud

- **Cloud Vision API**: analizar la foto y detectar monumentos, texto, objetos o elementos visuales relevantes.
- **Vertex AI / Gemini API**: generar la explicación turística, el enigma, las pistas y la validación de respuestas.
- **Cloud Translation API**: traducir la explicación o las pistas a otros idiomas si se añade esta funcionalidad.

- **Text-to-Speech API:** convertir la explicación turística en audio.
 - **Google Maps Platform:** mostrar mapa, ubicación, distancia y siguiente punto de la ruta.
 - **Cloud Firestore:** guardar puntuación, progreso, puntos desbloqueados y retos completados.
 - **Cloud Storage:** guardar imágenes subidas y audios generados si hace falta conservarlos.
 - **Cloud Run:** ejecutar el backend de forma segura sin exponer claves en el frontend.
-

3. Arquitectura propuesta

```

Usuario móvil
↓
Frontend PWA con Vue 3 + Ionic Vue
↓
Backend Node.js + Express en Cloud Run
↓
Cloud Storage + Cloud Vision API
↓
Vertex AI / Gemini API
↓
Translation API + Text-to-Speech API
↓
Google Maps Platform + Firestore
↓
Audioguía, enigma, progreso y siguiente punto

```

La separación entre frontend y backend es importante porque las claves y credenciales de Google Cloud **no deben estar en el navegador**. El frontend solo llama al backend, y el backend es quien usa las APIs privadas.

4. Estructura de carpetas recomendada

```

Mystery_Tourist_Lens/
  frontend/
    src/
      components/
      views/
      router/
      stores/
      services/
      assets/
    public/
    package.json
    vite.config.ts

  backend/
    src/
      routes/
      services/
      controllers/
      config/
    package.json
    Dockerfile

  docs/
    prompts/

```

```
rutas-demo/  
resumen-proyecto.md
```

5. Organización del repositorio en GitHub

El proyecto se organizará en un único repositorio de GitHub para que las tres personas trabajen sobre la misma base de código. La estructura separa claramente frontend, backend, documentación y datos de prueba.

```
Mystery_Tourist_Lens/  
  README.md  
  .gitignore  
  .env.example  
  
  frontend/  
    README.md  
    package.json  
    vite.config.ts  
    index.html  
    public/  
      manifest.webmanifest  
      icons/  
    src/  
      main.ts  
      App.vue  
      router/  
      stores/  
      views/  
      components/  
      services/  
      assets/  
      styles/  
  
  backend/  
    README.md  
    package.json  
    Dockerfile  
    .env.example  
    src/  
      server.js  
      routes/  
      controllers/  
      services/  
      config/  
      middleware/  
  
  docs/  
    resumen-proyecto.md  
    tecnologias-y-reparto.md  
    prompts/  
    rutas-demo/  
    capturas/  
  
  demo-data/  
    route.demo.json  
    points.demo.json
```

sample-responses.json

Qué contiene cada carpeta

- **frontend/**: aplicación Vue 3 con Ionic Vue. Aquí trabaja principalmente la Persona 1.
- **backend/**: API Node.js + Express para conectar con Google Cloud. Aquí trabaja principalmente la Persona 2.
- **docs/**: documentación del proyecto, resumen, prompts, planificación y material para la presentación. La mantienen las tres personas.
- **demo-data/**: datos estáticos de la ruta demo, puntos de interés y respuestas de prueba. Aquí trabaja principalmente la Persona 3.
- **README.md**: explicación general del proyecto, cómo instalarlo, cómo ejecutarlo y qué hace cada parte.
- **.env.example**: plantilla de variables de entorno sin claves reales.
- **.gitignore**: evita subir node_modules, archivos .env, builds y ficheros temporales.

Reglas básicas para trabajar en GitHub

1. La rama principal será main.
2. Cada persona trabajará en una rama propia:
 - feature/frontend-pwa
 - feature/backend-api
 - feature/mapa-datos-juego
3. No se subirán claves privadas ni archivos .env reales.
4. Cada cambio importante se subirá mediante Pull Request.
5. Antes de mezclar cambios en main, otra persona revisará que el proyecto siga arrancando.
6. Los datos de prueba se guardarán en demo-data/ para que todos puedan trabajar con la misma ruta.
7. La documentación importante se guardará en docs/, no repartida por conversaciones o archivos sueltos.

Responsabilidad por carpetas

- **Persona 1**: responsable principal de frontend/.
 - **Persona 2**: responsable principal de backend/.
 - **Persona 3**: responsable principal de demo-data/, docs/prompts/ y docs/rutas-demo/.
 - **Todo el grupo**: responsable de mantener actualizado el README.md principal.
-

6. Endpoints recomendados del backend

- **GET /api/route**: devuelve la ruta demo con sus puntos de interés.
- **POST /api/analyze-image**: recibe una foto, la guarda si hace falta y la analiza con Cloud Vision API.
- **POST /api/generate-guide**: genera la explicación turística y el enigma con Gemini.
- **POST /api/generate-audio**: convierte la explicación en audio con Text-to-Speech.
- **POST /api/check-answer**: comprueba si la respuesta del usuario es correcta.
- **GET /api/progress/:userId**: obtiene el progreso del usuario.
- **POST /api/progress**: guarda puntuación, punto actual y retos completados.

Para el MVP se puede simplificar el flujo y juntar varias operaciones en un solo endpoint:

POST /api/process-photo

Este endpoint puede recibir la foto y devolver directamente:

- Lugar detectado.
- Texto de audioguía.
- URL o base64 del audio.
- Pregunta o enigma.
- Opciones de respuesta.
- Siguiente punto de la ruta.

7. Autenticación para el MVP

Para reducir complejidad, no se recomienda implementar login real en el MVP.

La opción más sencilla es crear un identificador anónimo en el frontend y guardarlo en `localStorage`:

```
localStorage -> mystery_user_id
```

Con ese identificador se puede guardar el progreso en Firestore sin crear pantallas de registro, login o recuperación de contraseña.

8. Reparto de trabajo por personas

Persona 1: Frontend y experiencia de usuario

Responsabilidad principal: crear la PWA mobile-first y hacer que la aplicación sea cómoda de usar desde el móvil.

Tecnologías que usará:

- Vue 3.
- Vite.
- TypeScript.
- Ionic Vue.
- Vue Router.
- Pinia.
- Axios.
- vite-plugin-pwa.
- HTML5 Camera/File Input.
- HTML5 Audio.

Tareas concretas:

1. Crear el proyecto frontend con Vue 3, Vite y TypeScript.
2. Instalar y configurar Ionic Vue para tener una interfaz móvil con aspecto de app.
3. Crear las pantallas principales:
 - Inicio.
 - Punto actual de la ruta.
 - Cámara o subida de foto.
 - Resultado de análisis.
 - Audioguía.
 - Enigma o pregunta.
 - Mapa.
 - Progreso y puntuación.
4. Configurar Vue Router para navegar entre pantallas.
5. Configurar Pinia para guardar:
 - Usuario anónimo.
 - Punto actual.
 - Puntuación.
 - Retos completados.
 - Datos temporales de la audioguía y el enigma.
6. Integrar captura o subida de imagen desde el móvil.
7. Enviar la imagen al backend usando Axios.
8. Mostrar estados de carga mientras el backend analiza la imagen y genera contenido.
9. Integrar el reproductor de audio para escuchar la explicación turística.
10. Mostrar el enigma y enviar la respuesta al backend.

11. Mostrar feedback visual si la respuesta es correcta o incorrecta.
12. Configurar la PWA:
 - Manifest.
 - Icono.
 - Nombre de la app.
 - Color principal.
 - Instalación en pantalla de inicio.
13. Cuidar el diseño responsive para que funcione bien en móvil.

Entregable de la Persona 1:

Una WebApp/PWA funcional donde el usuario pueda recorrer todo el flujo visual: inicio, foto, audioguía, enigma, mapa y progreso.

Persona 2: Backend y APIs de Google Cloud

Responsabilidad principal: crear el backend seguro que conecta el frontend con las APIs de Google Cloud.

Tecnologías que usará:

- Node.js.
- Express.
- TypeScript o JavaScript.
- Multer.
- CORS.
- dotenv.
- Google Cloud Run.
- Cloud Vision API.
- Vertex AI / Gemini API.
- Text-to-Speech API.
- Cloud Translation API.
- Cloud Storage.

Tareas concretas:

1. Crear el proyecto backend con Node.js y Express.
2. Configurar variables de entorno para credenciales, proyecto de Google Cloud y claves necesarias.
3. Crear los endpoints principales:
 - GET /api/health
 - POST /api/analyze-image
 - POST /api/generate-guide
 - POST /api/generate-audio
 - POST /api/check-answer
 - POST /api/process-photo
4. Recibir imágenes desde el frontend usando Multer.
5. Subir imágenes a Cloud Storage si se decide conservarlas.
6. Analizar imágenes con Cloud Vision API.
7. Crear prompts para Gemini que generen:
 - Explicación turística breve.
 - Enigma o pregunta tipo test.
 - Opciones de respuesta.
 - Respuesta correcta.
 - Pista opcional.
8. Generar audio con Text-to-Speech API.
9. Añadir traducción con Cloud Translation API si el MVP incluye varios idiomas.
10. Devolver al frontend respuestas JSON claras y estables.
11. Manejar errores:
 - Imagen no válida.

- Lugar no detectado.
 - Error de API.
 - Respuesta vacía de Gemini.
12. Preparar el backend para desplegarlo en Cloud Run.
 13. Documentar qué datos espera y devuelve cada endpoint.

Entregable de la Persona 2:

Un backend desplegable en Cloud Run que reciba fotos, llame a Google Cloud y devuelva al frontend audioguía, enigma, audio y resultado de validación.

Persona 3: Mapa, datos, juego e integración

Responsabilidad principal: preparar la ruta demo, el sistema de progreso, la lógica del juego y la integración final entre frontend y backend.

Tecnologías que usará:

- Google Maps Platform.
- Firestore.
- Google Cloud Console.
- Pinia, si necesita coordinar estado con frontend.
- Axios/Postman para probar endpoints.
- JSON para definir rutas demo.
- Prompts de Gemini.

Tareas concretas:

1. Definir una ruta demo cerrada de 3 o 4 puntos de interés.
2. Para cada punto, preparar:
 - Nombre.
 - Descripción base.
 - Coordenadas.
 - Imagen de prueba.
 - Enigma esperado.
 - Respuesta correcta.
 - Siguiente punto.
3. Configurar Google Maps Platform y obtener la clave necesaria para el mapa.
4. Integrar el mapa en la pantalla correspondiente junto con la Persona 1.
5. Mostrar:
 - Ubicación actual del usuario.
 - Punto actual.
 - Siguiente punto desbloqueado.
 - Distancia aproximada.
6. Diseñar la estructura de datos en Firestore:

```
users/
  {userId}/
    progress/
      currentPointId
      score
      completedChallenges
      unlockedPoints

routes/
  demo-route/
    points/
      {pointId}
```

7. Implementar o coordinar el guardado de progreso:
 - Punto desbloqueado.
 - Puntuación.
 - Retos completados.
 - Estado de la ruta.
8. Probar el flujo completo:
 - Foto.
 - Análisis.
 - Audioguía.
 - Enigma.
 - Validación.
 - Desbloqueo.
 - Mapa.
 - Guardado de progreso.
9. Ajustar prompts de Gemini para que las respuestas sean cortas, claras y útiles para una demo.
10. Preparar datos de prueba para evitar que la demo dependa de resultados imprevisibles.

Entregable de la Persona 3:

Una ruta demo estable con mapa, puntos de interés, progreso guardado, puntuación y flujo de juego probado de principio a fin.

9. Plan de desarrollo recomendado

Semana 1: Base del proyecto

- **Persona 1:** crear frontend Vue/Ionic, pantallas base y navegación.
- **Persona 2:** crear backend Express, endpoints base y conexión inicial con Google Cloud.
- **Persona 3:** definir ruta demo, puntos de interés, estructura Firestore y claves de Maps.

Semana 2: Integración principal

- **Persona 1:** conectar frontend con backend, integrar foto, audio y pantallas de enigma.
- **Persona 2:** conectar Vision, Gemini y Text-to-Speech con respuestas JSON reales.
- **Persona 3:** integrar mapa, progreso, puntuación y pruebas con imágenes reales.

Semana 3: Pulido y demo

- **Persona 1:** mejorar diseño móvil, PWA, estados de carga y feedback visual.
 - **Persona 2:** corregir errores, estabilizar endpoints y preparar despliegue en Cloud Run.
 - **Persona 3:** probar la demo completa, ajustar prompts, preparar datos de respaldo y presentación.
-

10. Comandos iniciales recomendados

Frontend

```
npm create vue@latest frontend
cd frontend
npm install
npm install @ionic/vue @ionic/vue-router ionicons
npm install pinia vue-router axios
npm install vite-plugin-pwa
npm install @googlemaps/js-api-loader
```


Backend

```
mkdir backend
cd backend
npm init -y
npm install express cors multer dotenv
npm install @google-cloud/vision
npm install @google-cloud/text-to-speech
npm install @google-cloud/translate
npm install @google-cloud/storage
npm install @google-cloud/firestore
npm install @google-cloud/vertexai
```

11. Decisión final

El stack recomendado para desarrollar el proyecto de forma cómoda y realista es:

Frontend:

Vue 3 + Vite + TypeScript + Ionic Vue + Pinia + Vue Router

Backend:

Node.js + Express desplegado en Google Cloud Run

Datos:

Cloud Firestore + Cloud Storage

APIs:

Cloud Vision API + Vertex AI/Gemini + Text-to-Speech + Google Maps Platform

Esta combinación permite repartir bien el trabajo entre tres personas, evita desarrollar una app nativa para Android/iOS y mantiene el MVP lo bastante simple para completarlo en pocas semanas.